ViBR: Automated Bug Replay from Video-based Reports Using Vision-Language Models

ANONYMOUS AUTHOR(S)

Bug reports play a critical role in software maintenance by helping users convey encountered issues to developers. Recently, GUI screen capture videos have gained popularity as a bug reporting artifact due to their ease of use and ability to retain rich contextual information. However, automatically reproducing bugs from such recordings remains a significant challenge. Existing methods often rely on fragile image-processing heuristics, explicit touch indicators, or pre-constructed UI transition graphs, which require non-trivial instrumentation and app-specific setup. This paper presents ViBR, a lightweight and fully automated approach that reproduces bugs directly from GUI recordings. Specifically, ViBR combines CLIP-based embedding similarity for action segmentation with Vision-Language Models (VLMs) for region-aware GUI state comparison and guided bug replay. Experimental results show that ViBR successfully reproduces 73.7% of bug recordings, significantly outperforming state-of-the-art baselines and ablation variants.

CCS Concepts: • **Software and its engineering → Software testing and debugging**.

Additional Key Words and Phrases: Android bug replay, vision language model, GUI recording

ACM Reference Format:
Anonymous Author(s). 2018. ViBR: Automated Bug Replay from Video-based Reports Using Vision-Language Models. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 22 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Handling bug reports is a fundamental task in software maintenance, with bug reproduction often being the first and most critical step toward effective localization and resolution. However, many bugs are encountered by non-technical users who may lack the expertise or motivation to craft clear, structured reports [16, 19]. Poorly written reports are frequently misunderstood by developers [18, 21, 27], leading to repetitive communication and delays in fixing the issue.

To lower the barrier to bug reporting, video-based bug recordings [30]—screen capture videos of issues—have gained traction as a more accessible and expressive alternative to traditional reports. This trend is also reflected in platforms like GitHub, which now support video uploads in issue reports to foster clearer communication and faster collaboration between users and developers [8]. First, it is easy to record the screen as there are many tools available [2, 7], some of which are even embedded in the operating system by default like iOS [5] and Android [6]. Second, GUI recording can retain rich runtime context such as environmental configurations, dynamic content, and user input parameters, hence it bridges the understanding gap between users and developers.

Despite their growing prevalence, reproducing bugs from GUI recordings remains a manual and error-prone process. Developers must watch the raw footage to infer user actions and involved GUI elements—an ambiguous and time-consuming task, especially when accounting for differences in device. As shown in the Fig. 1, the same folder selection interface appears with different resolutions,

Conference acronym 'XX, Woodstock, NY

ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

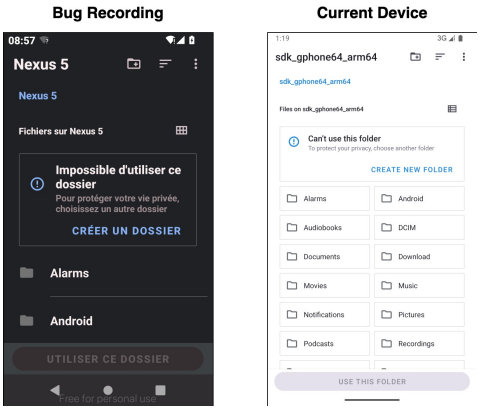


Fig. 1. GUI comparison between recording and device.

layouts, button placements, and background themes. Even small variations can hinder faithful bug reproduction on test devices.

While many existing techniques target bug reproduction from textual reports using pattern analysis and even state-of-the-art large language models (LLMs) [29, 31, 59, 71], these methods are not applicable to video-based reports, which lack structured step-by-step descriptions. Some recent efforts have explored video-based replay. For example, GIFdroid [30] aligns keyframes from GUI recordings to states in a pre-constructed UI Transition Graph (UTG) using pixel-level matching. However, this approach is brittle in real-world scenarios, where layout changes or dynamic GUI elements can easily break the alignment. V2S [17] leverages deep learning to detect touch indicators within the recording and replays those interactions accordingly. Yet, it requires users to enable touch indicators in advance when recording bugs, which imposes a heavy setup burden and limits its practicality for widespread adoption. Consequently, developing a lightweight, automated, and robust approach for replaying bugs from GUI recordings remains a critical need.

In this paper, we present ViBR, a novel framework for automatically performing **Vi**deo-based **B**ug **R**eplay without requiring app instrumentation, touch indicators, or pre-built UI graphs. Inspired by recent advances in Vision-Language Models (VLMs), like state-of-the-art GPT-4o, ViBR formulates bug reproduction as a multi-modal reasoning problem: given a recording and the current GUI state, the approach infers the next actionable step and executes it accordingly. Specifically, ViBR (1) segments the recording into distinct user interaction scenes by comparing CLIP embeddings of consecutive frames; (2) employs region-aware VLM-based reasoning to compare functionally equivalent GUI states between the recording and the target device; and (3) adaptively infers and replays the corresponding user actions on the device.

We conduct comprehensive evaluations of ViBR across the three phases of our approach. First, we evaluate our approach in segmenting the action scenes from 75 GUI recordings that are widely-studied in the prior studies, achieving up to 87%, 85%, and 86% in precision, recall, and F1-score, respectively. Second, we assess our approach in identifying functional consistency between the recorded and current GUI states. Compared with three state-of-the-art baselines and one ablation study, our approach achieves significantly better performance, i.e., 88% in precision, 92% in recall, and 89% in F1-score. In addition, we evaluate the ViBR in guiding bug replay and the results show that our approach can successfully reproduce 73.7% from real-world bug recordings, significantly improving over existing methods. The contributions of this paper are as follows:

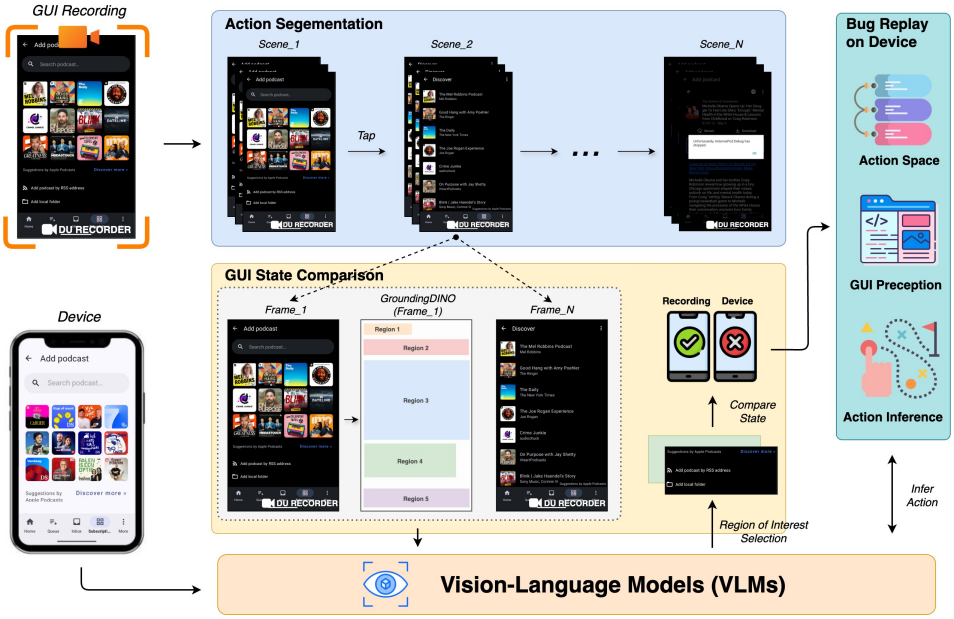


Fig. 2. The overview of ViBR.

- To the best of our knowledge, this is the first work that leverages Vision-Language Models for reasoning over GUI recordings, opening new possibilities for GUI-related engineering tasks.
- We introduce ViBR, a lightweight, fully automated framework that reproduces bugs directly from GUI recordings without requiring any heavy setups. The source code, dataset, and experimental results are in appendix[1].
- We perform a comprehensive evaluation on GUI recordings, demonstrating the effectiveness of ViBR compared to state-of-the-art baselines and ablation studies.

2 ViBR Approach

We present an automated approach for reproducing bugs from GUI recordings by segmenting the input video into user interaction scenes and conditionally replaying each action based on the current GUI state of the device. An overview of our approach is shown in Fig. 2 and comprises three key phases: (i) the *Action Segmentation* phase, where the GUI recording is divided into distinct scenes, each representing a single user action; (ii) the *GUI State Comparison* phase, which determines whether the GUI state associated with each scene matches the current screen on the device; and (iii) the *Bug Replay on Device* phase, which infers the intended user action and adaptively executes it on the device to reproduce the bug.

2.1 Action Segmentation

While recent work [63, 70] has explored Vision–Language Models (VLMs) for natural video segmentation, these models are not tailored to domain-specific video types such as GUI recordings, where transitions are driven not by natural scene changes but by discrete user interactions. To bridge this

[1]https://github.com/codereleaseww/ViBR

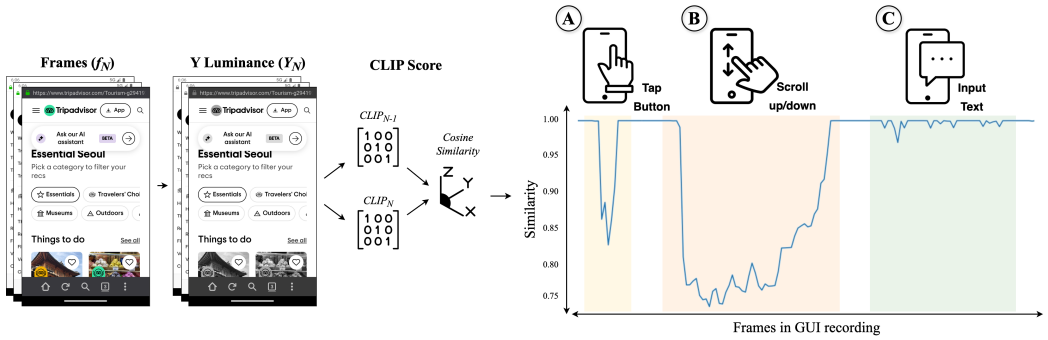


Fig. 3. An illustration of consecutive frame similarity.

gap, we adopt a signal-processing perspective, treating GUI recordings as ordered sequences of frames that encode state transitions initiated by user actions. Unlike natural-scene segmentation, which primarily focuses on detecting broad visual discontinuities, segmenting GUI recordings requires capturing fine-grained, user-driven state changes. Such transitions are often localized, triggered by actions like clicks, scrolling, or text input, yet they result in significant shifts in the interface layout. According to this insight, we segment GUI recordings into discrete user action scenes by analyzing the visual similarity between consecutive frames.

2.1.1 Consecutive Frame Comparison. Consider a GUI recording $\{f_0, f_1, .., f_{N-1}, f_N\}$, where f_N is the current frame and f_{N-1} is the previous frame, as shown in Fig. 3. Since raw video streams often contain visual noise (such as compression artifacts, color fluctuations, or subtle pixel jitter from screen refresh), we first reduce irrelevant variations by adopting a luminance-based representation. Inspired by a prior work on video segmentation [30], we employ Y-Difference (Y-Diff), a metric derived from the Y channel of the YUV color space, which is widely used in video compression due to its alignment with human visual perception—particularly the Y (luminance) component that dominates the recognition of structural and motion-related changes [23, 30, 54]. In practice, we convert each frame from RGB to YUV and extract the luminance mask Y_N for frame f_N.

Next, to capture both semantic and structural information of the frame, we leverage the lightweight semantic embedding model CLIP (Contrastive Language–Image Pre-training) [48]. We select CLIP because, unlike handcrafted similarity metrics that are highly sensitive to low-level perturbations (e.g., small shifts or noise), CLIP is trained on large-scale image–text pairs to learn a joint embedding space. This enables the embeddings to remain robust to such variations while encoding high-level perceptual and semantic information, yet still preserving meaningful pixel-level differences. Specifically, the luminance representations Y_N are first normalized and then passed through the CLIP image encoder, which produces high-dimensional embeddings $CLIP_N$ that preserve both the structural layout and the semantic content of the frame.

Finally, we compute the similarity between consecutive frames by applying the widely-used cosine similarity metric between their CLIP embeddings:

$$\text{Similarity}(f_{N-1}, f_N) = \frac{\langle \text{CLIP}(Y_{N-1}),\ \text{CLIP}(Y_N) \rangle}{\|\text{CLIP}(Y_{N-1})\| \cdot \|\text{CLIP}(Y_N)\|} \tag{1}$$

where $CLIP(Y_{N-1})$ and $CLIP(Y_N)$ denote the CLIP-encoded embeddings of the luminance masks of frames f_{N-1} and f_N, respectively. The resulting similarity score is normalized between 0 and 1,

where a higher value indicates strong visual and semantic similarity, while a lower value reflects substantial changes, typically corresponding to a GUI transition triggered by user interaction.

2.1.2 Frame Grouping. After computing pairwise CLIP scores, we group frames into atomic activity, e.g., user action. This procedure is essential because discrete user activities typically span multiple frames, requiring them to be grouped into coherent scenes that reflect each discrete action. To achieve this, we analyze the similarity series as shown in Fig. 3 to identify coherent segments. Consequently, we identify patterns for three primary interaction types: *Tap*, *Scroll*, and *Input*. Note that we focus on the most commonly-used actions for brevity in this paper, other actions could be extended by comparing the consecutive frame similarity.

(a) Tap: Characterized by a sharp drop in similarity as shown in Fig. 3-A, indicating a fast transition to a new screen, often triggered by clicking a button.

(b) Scroll: Typically begin with an initial drop, followed by gradual increases in similarity as shown in Fig. 3-B, reflecting the continuous nature of vertical scrolling where content shifts incrementally.

(c) Input: Involve oscillations in the similarity score as shown in Fig. 3-C, as the GUI reflects incremental changes while characters are typed. However, distinguishing input actions purely based on similarity is unreliable due to potential overlaps with tap-like transitions (e.g., focusing on a text field). Therefore, we enhance detection using Optical Character Recognition (OCR) [26] to identify the appearance of virtual keyboards. In detail, we extract OCR-detected characters from each frame and separate them into two sequences: ocr_{text} for alphabetic content and ocr_{num} for numeric content. Given OCR's imperfections in low-resolution or occluded text regions, we rely on the presence of keyboard-specific substrings to identify keyboard frames. For example, the detection of substrings such as *"qwert"* in ocr_{text} is indicative of an English keyboard, while patterns like *"123"* in ocr_{num} suggest a numeric keypad. We formally define a keyboard frame as one that satisfies:

$$frame = \begin{cases} \exists\{\text{qwert, asdfg, zxcvb}\} \in lowercase(ocr_{text}) \\ \exists\{123, 456, 789\} \in ocr_{num} \end{cases} \tag{2}$$

We deliberately avoid template-based keyboard matching, as the appearance of virtual keyboards can vary significantly across devices, due to custom themes, background modifications, screen sizes, and input methods.

2.2 GUI State Comparison

GUIs are inherently dynamic. Variations such as pop-up dialogs, layout shifts, and overlay configurations can introduce inconsistencies between the recorded environment and the current runtime state. These discrepancies often hinder accurate bug reproduction, as action scenes may not align with the live application interface, even on the same device. Therefore, we aim to verify whether the current GUI state matches the recorded scene before executing the corresponding user action.

A seemingly straightforward solution would be to apply traditional image similarity metrics to compare the recorded frame with the current screen. However, such methods fail to capture semantic equivalence. Visually distinct screens may support identical interactions (e.g., tapping "Android" folder button in Fig. 1), while visually similar screens may differ in functionality. This semantic gap renders pixel-level techniques insufficient for GUI reasoning.

To address this, we propose a region-guided, attention-based comparison framework that leverages Vision-Language Models (VLMs) to assess GUI state consistency with a focus on functionally relevant interaction targets. Our framework consists of three stages: interactive region detection, region of interest selection, and attention-driven state comparison.

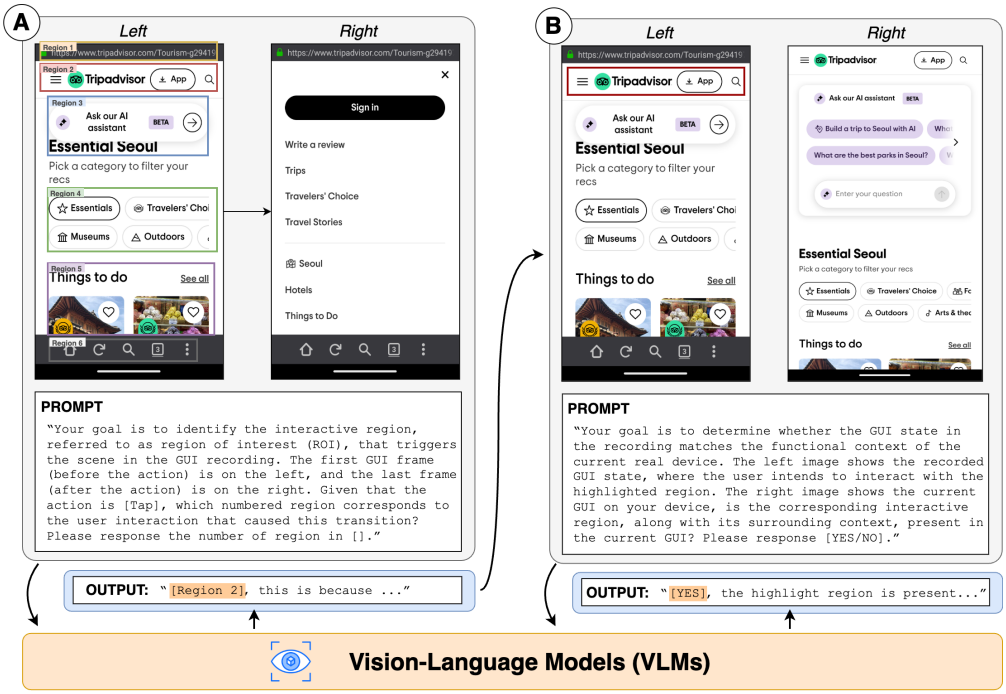


Fig. 4. The example of prompting GUI state comparison.

2.2.1 Interactive Region Detection. Since each frame in the GUI recording is a pure screenshot, we begin by identifying potential interaction regions. Note that rather than aiming to identify exact GUI elements, we focus on interactive regions, e.g., coarse-grained areas of the interface that likely contain actionable content. This design choice follows a coarse-to-fine paradigm, where coarse localization first narrows the search space, allowing for finer semantic reasoning.

To detect these regions, we use GroundingDINO [42], a state-of-the-art open-vocabulary object detector, which supports flexible, language-guided detection, making it well-suited for the diverse and dynamic nature of GUI interactive region detection. In detail, GroundingDINO contains three core components: a vision backbone, a language encoder, and a cross-modal decoder. The vision backbone is typically a transformer-based architecture pre-trained on large-scale image datasets. It extracts multi-scale visual features from the input GUI screenshot, capturing both global layout and fine-grained GUI element structures. The language encoder processes textual prompts that describe generic GUI interaction targets, such as "search bar", "layout", "button", or "text field". This encoder is based on CLIP, converting the textual queries into high-dimensional embeddings that semantically align with visual regions in the image. Finally, the cross-modal decoder then integrates visual and textual representations using attention mechanisms. It predicts bounding boxes and associated confidence scores for regions in the GUI that match the semantics of the interactive region. This process yields a set of candidate interactive regions from the frame of the action scene. These candidates serve as anchors for the subsequent reasoning and comparison steps. An example of interactive regions localized by GroundingDINO is shown in Fig. 4-A.

2.2.2 Region of Interest Selection. From the detected candidate regions, we identify the Region of Interest (ROI), e.g., the specific area corresponding to the user action. Accurate ROI selection

is critical to focus the comparison on relevant GUI elements while avoiding interference from unrelated GUI dynamics or background elements.

To enhance context, we incorporate temporal information by analyzing both the first and last frames of the action scene, representing the GUI before and after the user action, respectively. These two frames are merged into a dual-view composite image, preserving the full sequence context. Each candidate region from the first frame is labeled (e.g., "Region 1", "Region 2", etc.) and evaluated within this composite view. We then prompt the VLMs to identify which candidate region most likely corresponds to the user interaction by formulating a structured query as shown in Fig. 4-A. By grounding the reasoning in both spatial layout and semantic transitions, the VLMs select the true ROI. The output is a numeric index (e.g., *"Region 2"*), the potential region interacted with during the action scene.

2.2.3 Attention-Driven State Comparison. The final stage is to compare the GUI state captured in the action scene with the current GUI on the device. This comparison assesses whether the current interface is functionally consistent with the recorded state, ensuring that the next user action can be meaningfully and reliably replayed. In detail, we avoid traditional pixel-wise comparison and instead adopt a context-preserving, attention-guided approach using VLMs. The input to this prompt consists of two GUI screenshots: (1) the first frame of the action scene, in which the selected ROI is visually emphasized (e.g., by drawing a bounding box), and (2) the current GUI screen on the device where replay is intended. Both GUI screens are retained to preserve the entire interface context, allowing the model to reason about local alignment (region-level) while maintaining awareness of global consistency (screen-level). To facilitate the comparison, we design carefully crafted prompts that guide the VLM's attention toward the intended interaction target and contextual cues from the surrounding interface. An example prompt is shown in Fig. 4-B. To support interpretability and downstream control, we use binary-response formatting (*"[YES/NO]"*) in prompts, enabling confidence filtering based on the VLM's response probability.

2.3 Bug Replay on Device

Once GUI state consistency has been verified, we proceed to replay the recorded user action on the current device. If the current GUI state is deemed functionally equivalent to the recorded one, particularly with respect to the identified Region of Interest (ROI), the user action can be directly executed on the current interface. However, environmental differences (e.g., GUI content, layout shifts, or transient overlays) may cause divergence between the recorded and current states. In such cases, direct replay is infeasible, and guided exploration is required to bring the GUI into alignment with the expected precondition. The overarching goal remains the same: to navigate the current device screen, either with direct guidance from target actions or through exploration without explicit context, in order to ultimately reproduce the bug.

A key challenge is that the VLMs may lack domain-specific knowledge, that is, they do not possess an inherent understanding of how to execute mobile GUI-level actions. To bridge this gap, we design a prompt engineering technique to enable the VLMs to determine the next optimal action based on three core inputs: the executable action space, current GUI perception, and a structured prompt for intended interaction inference. Over successive reproduction, the VLMs iteratively navigates, adapts, and eventually replays the recorded action sequence to complete the bug replay process.

2.3.1 Action Space. To define the executable capabilities of the VLMs, we formalize a core action space in Table 1. This space includes three commonly used atomic actions: tap, scroll, and input. While more complex gestures such as pinch or multi-finger swipes do exist, they are relatively rare in practice. For clarity and generality, this work focuses on the most frequently observed

Table 1. Action space.

Action	Primitive	Description
tap	[tap] [element/back]	click on the element on the GUI screen or system backward to the previous screen.
scroll	[scroll] [direction]	navigate the screen vertically, up or down.
input	[input] [element] [value]	enter the value into GUI element field.
end	[end]	finish the replay task.

actions. Each action is represented using a structured format tailored to its interaction context, requiring distinct primitives to specify the necessary components. For example, the "tap" action requires either a target interactive element within the GUI (e.g., a button), represented as [tap] [element], or a system-level backward navigation, represented as [tap] [backward]. Similarly, the scroll action requires a directional specification (e.g., upward or downward), which we represent as [scroll] [direction]. When it comes to the "input" action, it involves entering a specified value into a particular field, and we structure this as [input] [element] [value]. Additionally, we introduce a special action, [end], which signals the completion of the replay sequence and denotes the end of the bug reproduction process.

2.3.2 Current GUI Perception. Accurate perception of the current GUI screen is essential for VLMs to understand the environment state. To this end, we construct a multimodal GUI representation by combining both visual capture and structural metadata. That is, we first acquire a visual screenshot of the current GUI screen using the Android Debug Bridge (ADB) screencast functionality [9], which provides a pixel-level rendering of the interface. Simultaneously, we extract the view hierarchy metadata through Android UIAutomator [10], which exposes the underlying XML structure of the GUI. This metadata contains detailed information about GUI components, including their types, positions, text content, visibility status, and interaction attributes (e.g., clickable, editable).

To extract relevant GUI elements from the XML structure, we perform a depth-first search (DFS) traversal on the view hierarchy tree. Starting from the root node, we iteratively explore each child node, resulting in a set of atomic GUI elements suitable for action grounding. Then, we apply a set-of-mark annotation process [68], in which each detected element is visually marked on the screenshot (e.g., with bounding boxes and labels). This augmented image, enriched with both visual and structural cues, serves as the perceptual input.

2.3.3 Action Inference. With the action space defined and the current GUI state perceived, we now leverage these inputs to construct structured prompts that guide the VLMs in inferring the action. This action inference process is inherently conditioned on the result of the prior GUI state consistency check (Section 2.2). Accordingly, we adopt a dynamic prompting strategy that tailors the action inference logic based on whether the current GUI state is consistent or inconsistent with the recorded action scene.

(a) Consistent State: When the current GUI matches the first frame of the recorded action scene, the objective is to faithfully replay the user action. To facilitate this, we construct a context-rich prompt that includes: (i) the first frame with the previously selected ROI highlighting the interaction target (Fig. 5-Left), (ii) the last frame of the scene representing the post-action state (Fig. 5-Middle), (iii) the current GUI screen (Fig. 5-Right), and (iv) the potential action (e.g., tap, scroll, input) obtained from action segmentation in Section 2.1. This composite prompt (as shown in Fig. 5-A) provides both temporal and spatial context, enabling the VLMs to infer the most plausible action that progresses the interface toward the intended postcondition.

Fig. 5. The example of prompting bug replay on device.

(b) Inconsistent State: When the current GUI state does not match the first frame of the recorded action scene, we should guide the interface to a compatible state before replaying the original user action. In this case, we construct a prompt that includes: (i) the first frame of the action scene, which serves as the target precondition (Fig. 5-Left), and (ii) the current GUI screen reflecting the present environment state (Fig. 5-Right). The VLMs is prompted to infer the most effective exploratory action that would transition the interface toward alignment with the recorded state, enabling the continuation of the replay sequence. An example prompt is shown in Fig. 5-B.

This process of inference and execution continues iteratively until the VLMs determine that the bug has been successfully reproduced, with the signal of the [end] action.

2.4 Implementation

Our ViBR is implemented as a fully automated tool for replaying bugs within mobile applications based on a GUI recording. We leverage the state-of-the-art GPT-4o from OpenAI [12] as the core VLM reasoning engine, and GroundingDINO [42] to perform object detection on GUIs throughout the replay process. To facilitate automated interpretation of the model's output, we define structured formatting instructions, enabling reliable parsing via regular expressions (e.g., []). For device-level execution, we use Genymotion [11] to allocate, launch, and manage virtual Android environments.

The GUI structure is extracted using Android UIAutomator [10], while Android Debug Bridge (ADB) [9] is used to perform input actions on the device.

3 Evaluation

In this section, we describe the procedure we used to evaluate ViBR in terms of its performance automatically. Since our approach consists of three main phases, we evaluate each phase of ViBR, including Action Segmentation (Section 2.1), GUI State Comparison (Section 2.2), and Bug Replay on Device (Section 2.3).

- **RQ1:** How accurate is our approach in segmenting the actions from GUI recordings?
- **RQ2:** How accurate is our approach in determining functional consistency in GUI states?
- **RQ3:** How effective is our approach in replaying the bug on device?

For **RQ1**, we evaluate the overall performance of our approach in action segmentation and compare it against state-of-the-art baselines. For **RQ2**, we assess the effectiveness of our approach in verifying GUI state consistency, determining whether the current on-device GUI matches the recorded state in the action scene. For **RQ3**, we examine the capability of our approach to successfully replay bugs on the device.

3.1 RQ1: Performance of Action Segmentation

3.1.1 Experimental Setup. To answer RQ1, we first evaluate the effectiveness of our approach (Section 2.1) in accurately segmenting user action scenes from GUI recordings. To ensure a diverse and unbiased dataset, we collect recordings from three existing open-source datasets: (i) the crash bug reproduction dataset from Themis [52]; (ii) the evaluation suite of GIFdroid [30]; and (iii) the study on Android GUI recording V2S [17]. Since some GUI recordings overlap across these datasets, we remove duplicates originating from the same issue repository. In total, we collect a dataset of 75 unique GUI recordings, yielding 638 action scenes, averaging 8.07 scenes per recording.

3.1.2 Metrics. We evaluate segmentation performance using three standard scene boundary detection metrics: precision, recall, and F1-score. A predicted scene boundary is considered correct if it falls within a tolerance window of ±5 frames [30] from the ground-truth boundary. Precision is defined as the proportion of predicted action scenes that are correctly matched to ground truth scenes:

$$precision = \frac{\#Correctly\ predicted\ action\ scenes}{\#All\ predicted\ action\ scenes}$$

Recall is defined as the proportion of all action scenes that are correctly detected by the predictions:

$$recall = \frac{\#Correctly\ predicted\ action\ scenes}{\#All\ action\ scenes}$$

F1-score is the harmonic mean of precision and recall, which combine both of the two metrics above.

$$F1 - score = \frac{2 \times precision \times recall}{precision + recall}$$

For all metrics, a higher value represents better performance.

3.1.3 Baselines. We compare our method against five state-of-the-art approach, which are widely used for video segmentation. *PySceneDetect* [4] is a practical tool that detects scene boundaries in videos based on threshold-based changes in visual content, such as frame-wise histogram differences. *Hecate* [50] is a tool developed by Yahoo that estimates frame quality on the image aesthetics and clusters them into scenes. *TransNetV2* [51] is a deep learning model trained on natural videos, using temporal convolutions to detect scene transitions. *GPT-4o* [12] is a multimodal large language

Table 2. Performance comparison for action segmentation.

Method	Precision	Recall	F1-score
PySceneDetect [4]	0.42	0.55	0.48
Hecate [50]	0.20	0.26	0.23
TransNetV2 [51]	0.33	0.56	0.41
GPT-4o [12]	0.25	0.55	0.34
GIFdroid [30]	0.81	0.84	0.82
ViBR	**0.87**	**0.85**	**0.86**

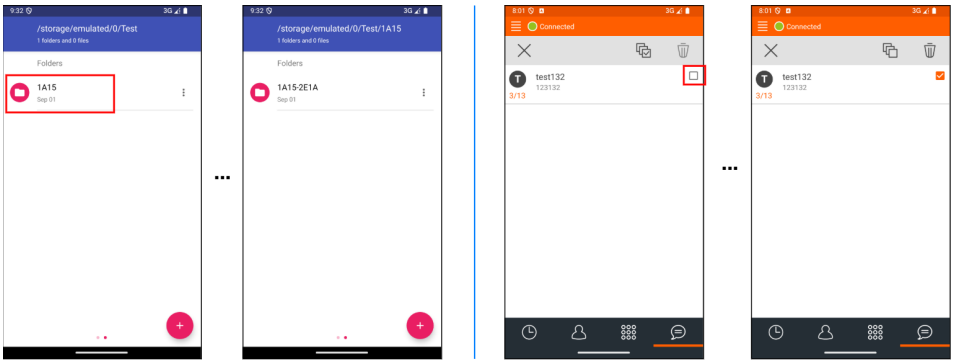


Fig. 6. Examples of failures cases of state-of-the-art baseline GIFdroid in action segmentation.

model prompted to output semantic scene boundaries from video input. *GIFDroid* [30] introduces an image-processing method by using the structural similarity index (SSIM) to segment GUI recordings into user actions.

3.1.4 Results. Table 2 presents the overall performance comparison across all baseline methods. Our approach significantly outperforms all baselines, achieving improvements of 7.4% in precision, 1.2% in recall, and 4.9% in F1-score over the best-performing baseline (*GIFdroid*). *GPT-4o* exhibits a lower performance, with a precision of 0.25, recall of 0.55, and F1-score of 0.34. While *GPT-4o* demonstrates strong semantic understanding in many tasks, it struggles with frame-level segmentation, primarily due to its reliance on high-level abstraction that lacks the granularity necessary for accurately identifying scene boundaries—particularly when user actions involve minor but functionally significant interface changes. This limitation is compounded by ambiguities in prompt interpretation and the definition of user action scenes, leading to underperformance in action segmentation tasks.

The deep learning-based baseline, *TransNetV2*, achieves an average precision of 0.33, recall of 0.56, and F1-score of 0.41. This relatively low performance is likely due to the domain discrepancy between the natural scenes on which *TransNetV2* was trained and the artificial nature of GUI recordings. Unlike natural scenes, which contain real-world elements such as humans, animals, and plants, GUI recordings involve static, highly structured content with specific rendering processes that it fails to generalize to effectively.

Traditional image-processing methods, such as *PySceneDetect* and *Hecate*, achieve relatively low F1-scores of 0.48 and 0.23, respectively. This is because these methods rely on low-level visual heuristics, e.g., histogram differences or aesthetic scoring, that are not well-suited to the subtle and

fine-grained transitions present in GUI recordings. As a result, they are less robust in handling the visual intricacies of GUIs. Among the baselines, *GIFdroid* performs best, achieving 0.81 precision, 0.84 recall, and 0.82 F1-score. However, since *GIFdroid* relies on SSIM as a perception metric, it remains limited when segmenting actions that involve large semantic differences but exhibit only minor pixel-level intensity differences. For example, as illustrated in Fig. 6, tapping a button triggers a transient overlay transition with low intensity variance: the new screen partially inherits the layout of the previous one, resulting in consecutive frames that appear structurally similar but are semantically different. In contrast, our approach leverages CLIP-based embeddings, enabling it to capture both semantic and structural changes.

Despite the superior performance, our method still encounters segmentation errors in certain scenarios. First, over-segmentation may occur when there are delays in resource loading, e.g., one segment is produced for the GUI transition, and another for the delayed rendering of content triggered by the same user action. Second, dynamic GUI elements such as advertisements or video playback can produce continuous frame differences that are mistakenly interpreted as separate user actions. These challenges highlight opportunities for future improvement, such as integrating motion analysis or adaptive temporal smoothing to reduce false splits.

3.2 RQ2: Performance of GUI State Comparison

3.2.1 Experimental Setup. To address RQ2, we evaluate the effectiveness of our method (as described in Section 2.2) in accurately determining functional consistency between the GUI state in each action scene and the current on-device GUI. Unlike existing datasets that focus on static screen similarity, our evaluation emphasizes functionality-aware comparison, e.g., determining whether the one GUI can support the same user interaction as the other, even if the appearance differs.

To construct the evaluation dataset, we build upon our experimental dataset described in Section 3.1.1, which contains 75 GUI recordings. During the reproduction of these recordings on the device, we manually verify whether the action could still be performed and annotate each pair of recorded and current GUI states with a binary label (consistent or inconsistent), which serves as ground truth. This process yields 463 GUI state comparisons, consisting of 352 consistent and 111 inconsistent cases.

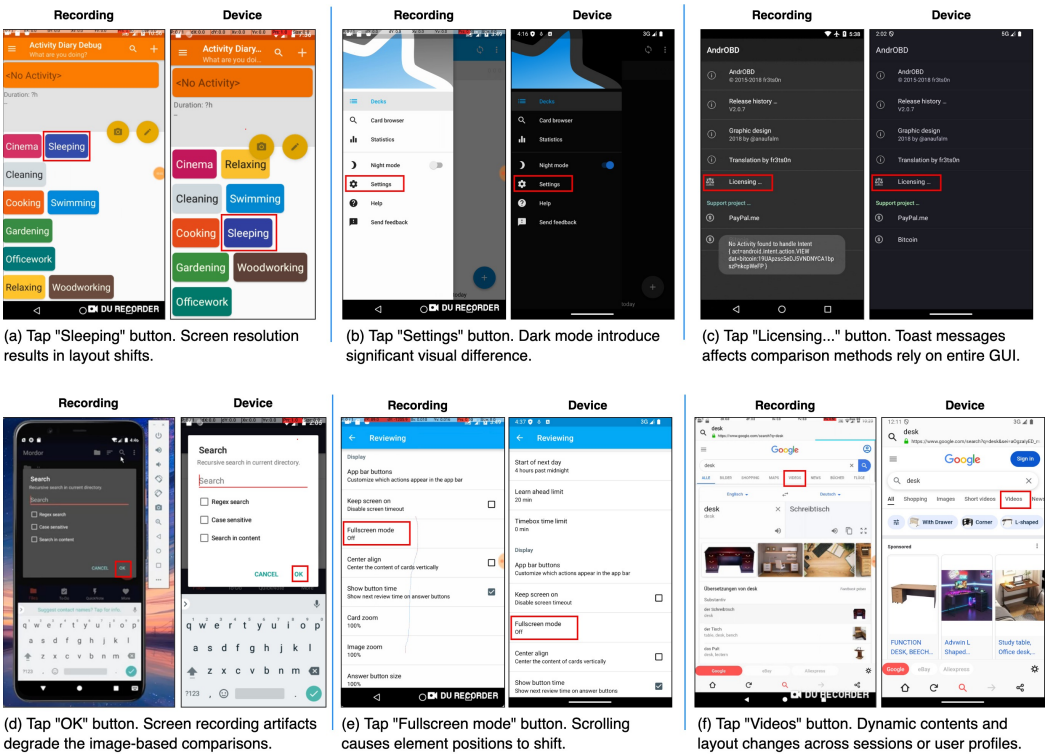
3.2.2 Metrics. We adopt the same evaluation metrics as in Section 3.1.2: precision, recall, and F1-score. In this context, a prediction is correct if the model's label (consistent or inconsistent) matches the ground-truth annotation for a given GUI state pair. These metrics evaluate how accurately the model captures functional alignment between recorded and live GUI states.

3.2.3 Baselines. We compare our method against four commonly used image comparison techniques, including one pixel-level (absolute differences *ABS* [62]), one structural-level (*SIFT* [43]), one perceptual-level (*SSIM* [60]), and one sematic-level (*CLIP* [48]) . Due to the page limit, we omitted the details of these well-known methods. Note that these methods only calculate similarity scores without explicit binary classifications of GUI comparison; thus, we adopt a threshold of 0.95 to determine the same GUI state. In addition, we include one ablated version of our approach to assess the effectiveness of key design components. The *VLM-only* variant removes the region-guided prompting mechanism and instead compares full-frame screenshots using a vision-language model directly. This baseline allows us to isolate the contribution of region-level interaction targeting.

3.2.4 Results. Table 3 summarizes the performance of all methods. Our approach consistently outperforms all baselines, achieving a 46.7% improvement in precision, 43.8% in recall, and 43.5% in F1-score compared to the best-performing baseline, SSIM. These results demonstrate the strength

Table 3. Performance of GUI state comparison.

Method	Precision	Recall	F1-score
ABS [62]	0.40	0.35	0.37
SIFT [43]	0.52	0.50	0.51
SSIM [60]	0.60	0.64	0.62
CLIP [48]	0.61	0.53	0.57
VLM-only	0.75	0.71	0.73
ViBR	**0.88**	**0.92**	**0.89**



(a) Tap "Sleeping" button. Screen resolution results in layout shifts.

(b) Tap "Settings" button. Dark mode introduce significant visual difference.

(c) Tap "Licensing..." button. Toast messages affects comparison methods rely on entire GUI.

(d) Tap "OK" button. Screen recording artifacts degrade the image-based comparisons.

(e) Tap "Fullscreen mode" button. Scrolling causes element positions to shift.

(f) Tap "Videos" button. Dynamic contents and layout changes across sessions or user profiles.

Fig. 7. Examples of failure cases of baselines in GUI comparison.

of our functionality-aware, VLM-driven comparison method over traditional visual similarity techniques.

Several factors contribute to this improvement. First, GUI environments are dynamic, and differences in screen resolution between the recording and test device can cause layout shifts, which confuse pixel- or structural-based methods such as *ABS* or *SIFT*. For instance, in Fig. 7(a), small scaling changes result in element misalignment, leading traditional methods to incorrectly classify the states as inconsistent. Second, cosmetic settings such as dark mode, font scaling, or language localization (Fig. 7(b)) can introduce significant visual differences without affecting interaction semantics. Third, non-functional artifacts such as toast messages or status banners (Fig. 7(c)) may appear during replay, misleading methods that depend on entire GUI matching. Fourth, screen

recording artifacts (e.g., emulator container in Fig. 7(d), watermarks from third-party tools, etc.) can further degrade the performance of image-based comparisons.

Our ablation study further validates the contribution of region-guided prompting. The *VLM-only* variant, which uses the entire GUI screenshots without localized prompts, does not perform well, particularly in recall (0.71 vs. 0.92), indicating its limited ability to focus on the semantic relevant interaction region. Compared to *VLM-only*, our approach achieves an improvement of 17.3%, 29.6%, and 21.9% for precision, recall, and F1-score, respectively. These improvements demonstrate the benefit of guiding the model's attention toward the target region, especially under visual inconsistencies. For example, in Fig. 7(e), layout shifts caused by scrolling result in position changes of the target "Fullscreen mode" button. Similarly, in Fig. 7(f), dynamic GUI changes between sessions alter the surrounding context of the target "Videos" tab. *VLM-only* fails to recognize these as functional consistent state due to reliance on holistic matching, whereas our region-guided prompt enables robust matching by reasoning over the local interaction semantics.

3.3 RQ3: Performance of Bug Replay

3.3.1 Experimental Setup. To answer RQ3, we evaluate the ability of our approach (Section 2.3) to effectively replay bugs on real devices using GUI recordings. We build upon the dataset described in Section 3.1.1, which originally contains 79 GUI recordings. However, many recordings are no longer reproducible due to several challenges. First, many real-world bugs have already been fixed in newer app versions, and retrieving the exact historical versions required for reproduction is often infeasible. Second, some bug scenarios, particularly in financial or social applications, require sensitive credentials, authenticated sessions, or hardware-specific environments beyond the scope of this study. Third, a subset of bug recordings lacks visible failure indicators (e.g., crash dialogs or error messages), making it difficult to confirm successful reproduction. Therefore, we manually replay each GUI recording on a test device to verify that the bug could still be reproduced. After filtering out unreproducible cases, we obtain a final dataset of 38 GUI recordings for experimentation.

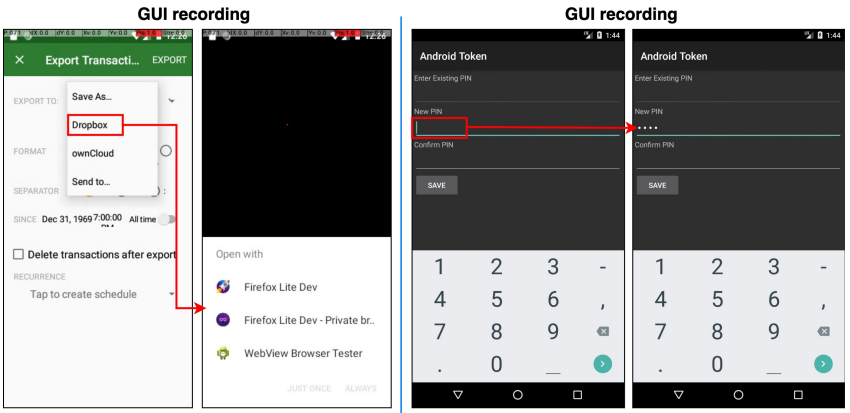
3.3.2 Metrics. We evaluate performance using two metrics: reproducibility and execution time. Reproducibility measures whether the approach can successfully execute the user actions in the GUI recording and reach the same buggy state. A higher score reflects greater robustness in replicating bug-triggering behaviors. Execution time captures the total runtime of the replay pipeline, including action segmentation, GUI state comparison, and action execution. The less time it takes, the more efficiently the method can reproduce the bugs.

3.3.3 Baselines. We compare our approach against two state-of-the-art bug replay techniques. *V2S* [17] is a visual record-and-replay method that detects user actions by identifying touch indicators in video frames using deep learning and then converting them into replayable scripts. *GIFdroid* [30] integrates image-processing with static program analysis that maps video keyframes to app states in a UI Transition Graph (UTG), enabling replay by matching visual scenes to graph transitions. To support GIFdroid, we first run Droidbot [39], an automated UI exploration tool, for 1 hour to collect the required UTG for each target app. This duration was selected based on prior findings [61] indicating that 1 hour provides sufficient time to achieve reasonable app coverage in exploration.

3.3.4 Results. Table 4 summarizes the performance of all methods. Our approach achieves an average reproducibility rate of 73.7% within 289.1 seconds, significantly outperforming both *V2S* and *GIFdroid*. This improvement is largely due to the robustness of our method against the types of inconsistencies discussed in Section 3.2.4, such as resolution mismatches, dynamic UI content,

Table 4. Performance comparison for bug replay.

Method	Reproducibility	Execution time
V2S [17]	47.4%	904.3s
GIFdroid [30]	52.6%	346.3s
ViBR	**73.7%**	**289.1s**



(a) Tap "Dropbox" button. Lack of clear semantic correlation between GUI states.

(b) Input "1255". Lack of intermediate transition between GUI states.

Fig. 8. Examples of failures cases of our approach ViBR in bug replaying.

configuration variability, and recording artifacts. These factors often cause baseline methods to fail due to their reliance on fragile visual or structural assumptions.

In addition, a key advantage of our approach lies in its reduced reliance on auxiliary cues or complete structural knowledge. For example, *V2S* requires clearly visible touch indicators to infer user actions, an assumption that rarely holds in user-submitted bug recordings (e.g., from GitHub). As a result, it fails to handle a substantial portion (21.5%) of our dataset. *GIFdroid*, on the other hand, relies heavily on the completeness and accuracy of the UTG, which is often difficult to construct due to non-deterministic transitions, GUI complexity, and incomplete exploration, resulting in frequent replay failures. In contrast, our approach is lightweight without requiring instrumentation or app-specific configurations, harnessing the multi-modal reasoning capabilities of vision-language models to achieve functionality-aware GUI matching and robust action inference across diverse device environments and configurations.

Albeit the good performance of our approach, we still fail to replay certain cases. We manually check those failure cases and summarise two common causes. First, the lack of clear semantic correlation between GUI states. For instance, in Fig. 8(a), tapping the "Dropbox" button leads to a next screen with no clear semantic correlation with the original GUI, making it hard for the VLMs to infer the action. Second, as shown in Fig. 8(b), inputting a PIN (e.g., "1255") results in masked output ("****"), which prevents the VLMs from identifying the precise input without additional context about the intermediate transition. In the future, we plan to sample intermediate frames within each scene. While this introduces additional computational overhead, it can enrich contextual understanding, particularly in cases involving semantically ambiguous or visually obfuscated transitions. This temporal augmentation may help VLMs more effectively associate cause-effect pairs across challenging UI changes.

4 Threats To Validity

In evaluating our approach, one internal validity arises from the inherent stochasticity of VLMs, which may produce slightly different outputs across runs given the same prompt. Consequently, this variability may introduce fluctuations in performance metrics. To mitigate this, we executed all VLM-based approaches, including our approach and VLM-related baselines, three times and reported the best performance. While we do observe a small number of hallucinations and instances of randomness, they account for <2% of all experiments and we therefore believe they do not significantly impact our overall conclusions.

Another potential source of internal bias stems from the manual labeling of ground-truth in our experimental dataset (Section 3). To ensure annotation reliability, all annotators involved in the labeling process had at least 1.5 years of experience in Android app development and testing. To further reduce subjectivity, all ground-truth labels were cross-validated and subjected to a final random audit to catch any potential inconsistencies or errors.

As for external validity, the primary threat concerns the representativeness of the dataset used to evaluate our approach. To address this, we selected GUI recordings from three prior studies in the domain of video-based bug reproduction, which together form a diverse and unbiased dataset. These bug reports were originally sourced from real-world applications, thus ensuring that our evaluation setting reflects practical and realistic testing scenarios. Further empirical studies on larger and more heterogeneous datasets may required to confirm broader generalizability.

Another potential threat to validity lies in the generalizability of our approach to other VLMs, such as Gemini, Claude, and Qwen. Note that `ViBR` is designed to be model-agnostic, with a primary focus on prompt engineering rather than reliance on a specific VLM. Recent studies [20, 31, 35, 69] often evaluate their model-agnostic methods using a single state-of-the-art model as a representative case. Similarly, our current experiments employ GPT-based models to establish a strong initial benchmark and to demonstrate the feasibility of VLM-driven approaches for GUI-based bug replay. In future work, we will systematically assess `ViBR`'s performance across a range of VLMs to further validate its general applicability.

5 Related Work

A growing body of tools has been dedicated to assisting in recording and replaying bugs in mobile and web apps. We review the related work in two main areas: 1) bug record and replay, and 2) vision-language models for software engineering.

5.1 Bug Record and Replay

Nurmuradov et al. [46] introduced a record-and-replay tool for Android applications that captures user interactions by rendering the device screen in a web browser and logging touch events. It generates heatmaps to visualize user interactions, enabling developers to replay and analyze usage patterns. Other tools such as ECHO [55], Reran [32], Barista [37], and Android Bot Maker [1] adopt program analysis techniques to record and replay user actions. However, these tools typically require heavy instrumentation or system-level frameworks, such as replaykit [3], troyd [36], or app-level modifications, which can impose significant overhead and are often too heavy for end users.

Several research have also focused on automating bug replay based on structured bug reports. For example, Fazzini et al. [29] proposed Yakusu, which combines program analysis with natural language processing (NLP) to synthesize executable test cases from bug descriptions. ReCDroid [71], introduced by Zhao et al., enhanced this idea by incorporating lexical knowledge to improve crash reproduction accuracy. However, these methods often overlook the temporal relationships among

reproduction steps (e.g., "after tapping X, open Y"), which are crucial for accurately modeling interaction flows. Liu et al. [40] addressed this limitation by proposing MaCa, an NLP and machine learning-based approach that normalizes S2R (steps-to-reproduce) instructions and extracts structured entities. Recent advances in large language models (LLMs) have inspired prompt-based methods for textual bug replay, including AdbGPT [31] and ReBL [59], which use few-shot learning and feedback prompting that significantly improve the performance. Despite their advancements, these techniques still heavily depend on the quality and consistency of natural language in bug reports [18, 27, 38], including formatting, vocabulary, and granularity.

A few studies [58] have leveraged visual information from bug recordings to assist replay. For instance, Bernal et al. [17] proposed V2S, a video-to-script system that uses deep learning to detect user actions based on touch indicators visible in the recording, translating them into executable test scripts. Similarly, Feng et al. [30] introduced GIFdroid, which applies image processing techniques to extract keyframes from screen recordings and map them to states in a GUI transition graph (UTG) for replay. In contrast, our approach offers a lightweight solution that does not require touch indicators or UTG collection. In detail, we leverage vision-language models to reason over both GUI appearance and interaction context, enabling functionality-aware action inference and robust replay across different device environments, making it more broadly applicable and easier to deploy in real-world scenarios.

Some works attempted to assist bug reporting process [53, 65, 66] and improve the bug report quality [45, 64, 67]. For example, Chaparro et al. [22] developed DeMIBuD to detect missing information from bug reports. Moran et al. [45] introduced Fusion, a web-based application that allows users to report the S2Rs graphically by selecting images of the GUI components and actions to reproduce the bugs. Similarly, Fazzini et al. [28] proposed a mobile application EBug, suggesting accurate reproduction steps while the users are writing them. More recently, Yang et al. [49] proposed an interactive chatbot system BURT to guide users to report essential bug report elements. These approaches are complementary to our approach, as they may increase bug recording quality at the time of reporting which in turn, could potentially improve the capabilities of VLMs' understanding.

5.2 Vision-Language Models for Software Engineering

Following the success of Large Language Models (LLMs) across a wide range of natural language processing tasks, researchers have increasingly explored their application to software engineering. Early efforts have primarily focused on code-related tasks [25, 34], such as automated code generation, completion, and translation. A notable example is Codex [13], the model behind GitHub Copilot, which demonstrated the ability to translate natural language comments into executable code, effectively reducing developer workload and narrowing the gap between user intent and implementation.

More recently, research has begun shifting toward multi-modal models, particularly Vision-Language Models (VLMs), which integrate visual and textual modalities to support richer forms of reasoning across tasks [24, 41], such as visual question answering, human action recognition, and embodied robotics. Despite their success in these areas, applications of VLMs to software engineering remain relatively nascent. One emerging direction lies in GUI-based software tasks, such as generating executable GUI code from screenshots. For example, Wan et al. [56] introduced MRWeb, which systematically explored prompting strategies, including zero-shot and chain-of-thought prompting, to improve contextual understanding and generation accuracy. Building on this idea, Wan et al. [57] proposed DCGen, a pipeline that segments GUI screenshots into components and then prompts VLMs to generate corresponding code snippets, which are reassembled into full-page implementations. Recently, self-refinement prompting has been investigated to further improve model outputs: models are instructed to generate a candidate solution, critique it, and

revise iteratively. For instance, Zhou et al. [72] proposed DeclarUI, which enhances GUI code generation quality through this refinement loop.

Beyond GUI code generation, VLMs have also unlocked new opportunities in GUI automation and software testing, where they are increasingly explored for their ability to perceive, reason about, and interact with user interfaces. Claude [15], for instance, introduced the concept of a "computer use agent", a class of agents that operate directly over GUIs using vision-language reasoning rather than API hooks. Extending this concept, frameworks such as CogAgent [33], WebLINX [44], and UX-LLM [47] apply VLMs to automate GUI navigation, execute predefined tasks, and conduct usability testing in web or desktop environments. These systems represent some of the earliest attempts to exploit the perceptual and semantic capabilities of VLMs for GUI interaction tasks that were previously dominated by heuristic or API-level methods. Additionally, Feng et al. [14] recently proposed a taxonomy of Autonomous GUI Automation Levels, offering a conceptual framework to categorize GUI agents based on their degree of autonomy and intelligence.

Despite these promising developments, most existing approaches are limited to reasoning over single static GUI screenshots, which restricts their ability to model temporal dynamics and user workflows that unfold over time. In contrast, our work is the first to leverage VLMs in the context of GUI recordings, treating them as sequences of visual frames to reason about user intent and interaction across time. By enabling both bug interpretation and replay from screen recordings, our approach extends the scope of VLMs beyond static perception to temporal, functionality-aware GUI understanding, bridging a critical gap in automated debugging and offering a new paradigm for VLM-based software testing.

6 Conclusion and Future Work

GUI recordings in video-based bug reports are becoming increasingly popular due to their ease of creation and rich contextual information. This paper introduces `ViBR`, a lightweight and fully automated approach for reproducing bugs on devices using GUI recordings. Unlike prior methods that depend on pixel-level similarity, explicit touch indicators, or pre-collected UI transition graphs, `ViBR` leverages CLIP-based embeddings together with the semantic reasoning capabilities of Vision-Language Models (VLMs) like GPT-4o to understand both visual context and user intent over time. In detail, we first segment GUI recordings into discrete scenes representing individual user interactions. We then propose a novel attention-guided prompting strategy combined with region-level reasoning to compare the GUI state between each recorded scene and the current device screen. Based on the comparison, our approach infers and executes the appropriate sequence of actions to robustly guide the bug replay process. Through extensive experiments, `ViBR` significantly outperforms state-of-the-art baselines and ablation variants, demonstrating its effectiveness in accurately reproducing bugs.

While `ViBR` demonstrates the feasibility of leveraging VLMs for interpreting and replaying GUI recordings, several avenues remain open for future exploration. First, we plan to enhance action inference by incorporating intermediate animation frames between UI transitions, rather than relying solely on pre- and post-action frames. This would provide richer temporal context and improve inference accuracy. Second, we aim to expand the action space supported by our approach to include more complex high-level interactions, such as pinch-in gestures and multi-touch inputs, thereby broadening the applicability of `ViBR` to other user interfaces. Finally, beyond debugging, the broader paradigm of reasoning over GUI recordings opens possibilities for new application domains such as usability testing, automated tutorial generation, and accessibility evaluation. By modeling user interaction patterns over time, `ViBR` could evolve into a general-purpose framework for understanding, simulating, and improving user–software interaction.

7 Data Availability

All the source code, dataset, and experimental results are available at https://github.com/codereleaseww/ViBR for replication and future research.

References

[1] 2021. Bot Maker for Android - Apps on Google Play. https://play.google.com/store/apps/details?id=com.frapeti.androidbotmaker.

[2] 2021. BugClipper. https://bugclipper.com/.

[3] 2021. Command line tools for recording, replaying and mirroring touchscreen events for Android. https://github.com/appetizerio/replaykit.

[4] 2021. Python and OpenCV-based scene cut/transition detection program & library. https://github.com/Breakthrough/PySceneDetect.

[5] 2021. Record the screen on your iPhone, iPad, or iPod touch. https://support.apple.com/en-us/HT207935.

[6] 2021. Take a screenshot or record your screen on your Android device. https://support.google.com/android/answer/9075928?hl=en.

[7] 2021. TestFairy. https://www.testfairy.com/.

[8] 2021. Video uploads now available across GitHub. https://github.blog/news-insights/product-news/video-uploads-available-github/.

[9] 2023. Android Debug Bridge (adb) - Android Developers. https://developer.android.com/studio/command-line/adb.

[10] 2023. Android Uiautomator2 Python Wrapper. https://github.com/openatx/uiautomator2.

[11] 2023. Genymotion – Android Emulator for app testing. https://www.genymotion.com/.

[12] 2023. Introducing ChatGPT. https://chat.openai.com/.

[13] 2023. OpenAI Codex. https://openai.com/blog/openai-codex.

[14] 2025. How Smart Is Your GUI Agent? A Framework for the Future of Software Interaction. https://medium.com/@charliefeng1996/how-smart-is-your-gui-agent-89b1d6c0fe92.

[15] 2025. Introducing computer use, a new Claude 3.5 Sonnet, and Claude 3.5 Haiku. https://www.anthropic.com/news/3-5-models-and-computer-use.

[16] Jorge Aranda and Gina Venolia. 2009. The secret life of bugs: Going past the errors and omissions in software repositories. In *2009 IEEE 31st International Conference on Software Engineering*. IEEE, 298–308.

[17] Carlos Bernal-Cárdenas, Nathan Cooper, Kevin Moran, Oscar Chaparro, Andrian Marcus, and Denys Poshyvanyk. 2020. Translating video recordings of mobile app usages into replayable scenarios. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*. 309–321.

[18] Nicolas Bettenburg, Sascha Just, Adrian Schröter, Cathrin Weiss, Rahul Premraj, and Thomas Zimmermann. 2008. What makes a good bug report?. In *Proceedings of the 16th ACM SIGSOFT International Symposium on Foundations of software engineering*. 308–318.

[19] Nicolas Bettenburg, Rahul Premraj, Thomas Zimmermann, and Sunghun Kim. 2008. Extracting structural information from bug reports. In *Proceedings of the 2008 international working conference on Mining software repositories*. 27–30.

[20] Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. 2024. Repairagent: An autonomous, llm-based agent for program repair. *arXiv preprint arXiv:2403.17134* (2024).

[21] Oscar Chaparro, Carlos Bernal-Cárdenas, Jing Lu, Kevin Moran, Andrian Marcus, Massimiliano Di Penta, Denys Poshyvanyk, and Vincent Ng. 2019. Assessing the quality of the steps to reproduce in bug reports. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 86–96.

[22] Oscar Chaparro, Jing Lu, Fiorella Zampetti, Laura Moreno, Massimiliano Di Penta, Andrian Marcus, Gabriele Bavota, and Vincent Ng. 2017. Detecting missing information in bug descriptions. In *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*. 396–407.

[23] Hu Chen, Mingzhe Sun, and Eckehard Steinbach. 2009. Compression of Bayer-pattern video sequences using adjusted chroma subsampling. *IEEE transactions on circuits and systems for video technology* 19, 12 (2009), 1891–1896.

[24] Guangzhao Dai, Xiangbo Shu, Wenhao Wu, Rui Yan, and Jiachao Zhang. 2024. GPT4Ego: unleashing the potential of pre-trained models for zero-shot egocentric action recognition. *IEEE Transactions on Multimedia* (2024).

[25] Yinlin Deng, Chunqiu Steven Xia, Haoran Peng, Chenyuan Yang, and Lingming Zhang. 2022. Fuzzing Deep-Learning Libraries via Large Language Models. *arXiv preprint arXiv:2212.14834* (2022).

[26] Yuning Du, Chenxia Li, Ruoyu Guo, Cheng Cui, Weiwei Liu, Jun Zhou, Bin Lu, Yehua Yang, Qiwen Liu, Xiaoguang Hu, et al. 2021. PP-OCRv2: Bag of Tricks for Ultra Lightweight OCR System. *arXiv preprint arXiv:2109.03144* (2021).

[27] Mona Erfani Joorabchi, Mehdi Mirzaaghaei, and Ali Mesbah. 2014. Works for me! characterizing non-reproducible bug reports. In *Proceedings of the 11th Working Conference on Mining Software Repositories*. 62–71.

[28] Mattia Fazzini, Kevin Patrick Moran, Carlos Bernal-Cardenas, Tyler Wendland, Alessandro Orso, and Denys Poshyvanyk. 2022. Enhancing Mobile App Bug Reporting via Real-time Understanding of Reproduction Steps. *IEEE Transactions on Software Engineering* (2022).

[29] Mattia Fazzini, Martin Prammer, Marcelo d'Amorim, and Alessandro Orso. 2018. Automatically translating bug reports into test cases for mobile apps. In *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 141–152.

[30] Sidong Feng and Chunyang Chen. 2022. GIFdroid: automated replay of visual bug reports for Android apps. In *Proceedings of the 44th International Conference on Software Engineering*. 1045–1057.

[31] Sidong Feng and Chunyang Chen. 2024. Prompting is all you need: Automated android bug replay with large language models. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–13.

[32] Lorenzo Gomez, Iulian Neamtiu, Tanzirul Azim, and Todd Millstein. 2013. Reran: Timing-and touch-sensitive record and replay for android. In *2013 35th International Conference on Software Engineering (ICSE)*. IEEE, 72–81.

[33] Wenyi Hong, Weihan Wang, Qingsong Lv, Jiazheng Xu, Wenmeng Yu, Junhui Ji, Yan Wang, Zihan Wang, Yuxiao Dong, Ming Ding, et al. 2024. Cogagent: A visual language model for gui agents. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 14281–14290.

[34] Qing Huang, Yanbang Sun, Zhenchang Xing, Min Yu, Xiwei Xu, and Qinghua Lu. 2023. API Entity and Relation Joint Extraction from Text via Dynamic Prompt-tuned Language Model. *arXiv preprint arXiv:2301.03987* (2023).

[35] Syed Fatiul Huq, Mahan Tafreshipour, Kate Kalcevich, and Sam Malek. 2024. Automated Generation of Accessibility Test Reports from Recorded User Transcripts. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, 534–546.

[36] Jinseong Jeon and Jeffrey S Foster. 2012. *Troyd: Integration testing for android.* Technical Report.

[37] Andrew J Ko and Brad A Myers. 2006. Barista: An implementation framework for enabling new tools, interaction techniques and views in code editors. In *Proceedings of the SIGCHI conference on Human Factors in computing systems*. 387–396.

[38] A Gunes Koru and Jeff Tian. 2004. Defect handling in medium and large open source projects. *IEEE software* 21, 4 (2004), 54–61.

[39] Yuanchun Li, Ziyue Yang, Yao Guo, and Xiangqun Chen. 2017. Droidbot: a lightweight ui-guided test input generator for android. In *2017 IEEE/ACM 39th International Conference on Software Engineering Companion (ICSE-C)*. IEEE, 23–26.

[40] Hui Liu, Mingzhu Shen, Jiahao Jin, and Yanjie Jiang. 2020. Automated classification of actions in bug reports of mobile apps. In *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 128–140.

[41] Li Liu, Diji Yang, Sijia Zhong, Kalyana Suma Sree Tholeti, Lei Ding, Yi Zhang, and Leilani Gilpin. 2024. Right this way: Can VLMs Guide Us to See More to Answer Questions? *Advances in Neural Information Processing Systems* 37 (2024), 132946–132976.

[42] Shilong Liu, Zhaoyang Zeng, Tianhe Ren, Feng Li, Hao Zhang, Jie Yang, Qing Jiang, Chunyuan Li, Jianwei Yang, Hang Su, et al. 2024. Grounding dino: Marrying dino with grounded pre-training for open-set object detection. In *European Conference on Computer Vision*. Springer, 38–55.

[43] David G Lowe. 2004. Distinctive image features from scale-invariant keypoints. *International journal of computer vision* 60, 2 (2004), 91–110.

[44] Xing Han Lù, Zdeněk Kasner, and Siva Reddy. 2024. Weblinx: Real-world website navigation with multi-turn dialogue. *arXiv preprint arXiv:2402.05930* (2024).

[45] Kevin Moran, Mario Linares-Vásquez, Carlos Bernal-Cárdenas, and Denys Poshyvanyk. 2015. Auto-completing bug reports for android applications. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*. 673–686.

[46] Dmitry Nurmuradov and Renee Bryce. 2017. Caret-HM: recording and replaying Android user sessions with heat map generation using UI state clustering. In *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 400–403.

[47] Ali Ebrahimi Pourasad and Walid Maalej. 2024. Does GenAI Make Usability Testing Obsolete? *arXiv preprint arXiv:2411.00634* (2024).

[48] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. 2021. Learning transferable visual models from natural language supervision. In *International conference on machine learning*. PmLR, 8748–8763.

[49] Yang Song, Junayed Mahmud, Ying Zhou, Oscar Chaparro, Kevin Moran, Andrian Marcus, and Denys Poshyvanyk. 2022. Toward interactive bug reporting for (android app) end-users. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 344–356.

[50] Yale Song, Miriam Redi, Jordi Vallmitjana, and Alejandro Jaimes. 2016. To click or not to click: Automatic selection of beautiful thumbnails from videos. In *Proceedings of the 25th ACM International on Conference on Information and Knowledge Management*. 659–668.

[51] Tomás Soucek and Jakub Lokoc. 2024. Transnet v2: An effective deep network architecture for fast shot transition detection. In *Proceedings of the 32nd ACM International Conference on Multimedia*. 11218–11221.

[52] Ting Su, Jue Wang, and Zhendong Su. 2021. Benchmarking automated gui testing for android against real-world bugs. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 119–130.

[53] Yuhui Su, Chunyang Chen, Junjie Wang, Zhe Liu, Dandan Wang, Shoubin Li, and Qing Wang. 2022. The Metamorphosis: Automatic Detection of Scaling Issues for Mobile Apps. In *37th IEEE/ACM International Conference on Automated Software Engineering*. 1–12.

[54] Ramadass Sudhir and Lt Dr S Santhosh Baboo. 2011. An efficient CBIR technique with YUV color space and texture features. *Computer Engineering and Intelligent Systems* 2, 6 (2011), 78–85.

[55] Yulei Sui, Yifei Zhang, Wei Zheng, Manqing Zhang, and Jingling Xue. 2019. Event trace reduction for effective bug replay of Android apps via differential GUI state analysis. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1095–1099.

[56] Yuxuan Wan, Yi Dong, Jingyu Xiao, Yintong Huo, Wenxuan Wang, and Michael R Lyu. 2024. Mrweb: An exploration of generating multi-page resource-aware web code from ui designs. *arXiv preprint arXiv:2412.15310* (2024).

[57] Yuxuan Wan, Chaozheng Wang, Yi Dong, Wenxuan Wang, Shuqing Li, Yintong Huo, and Michael Lyu. 2025. Divide-and-Conquer: Generating UI Code from Screenshots. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 2099–2122.

[58] Dingbang Wang, Zhaoxu Zhang, Sidong Feng, William GJ Halfond, and Tingting Yu. 2025. An Empirical Study on Leveraging Images in Automated Bug Report Reproduction. *arXiv preprint arXiv:2502.15099* (2025).

[59] Dingbang Wang, Yu Zhao, Sidong Feng, Zhaoxu Zhang, William GJ Halfond, Chunyang Chen, Xiaoxia Sun, Jiangfan Shi, and Tingting Yu. 2024. Feedback-driven automated whole bug report reproduction for android apps. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1048–1060.

[60] Shiqi Wang, Abdul Rehman, Zhou Wang, Siwei Ma, and Wen Gao. 2011. SSIM-motivated rate-distortion optimization for video coding. *IEEE Transactions on Circuits and Systems for Video Technology* 22, 4 (2011), 516–529.

[61] Wenyu Wang, Dengfeng Li, Wei Yang, Yurui Cao, Zhenwen Zhang, Yuetang Deng, and Tao Xie. 2018. An empirical study of android test generation tools in industrial cases. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*. 738–748.

[62] Craig Watman, David Austin, Nick Barnes, Gary Overett, and Simon Thompson. 2004. Fast sum of absolute differences visual landmark detector. In *IEEE International Conference on Robotics and Automation, 2004. Proceedings. ICRA'04. 2004*, Vol. 5. IEEE, 4827–4832.

[63] Yuetian Weng, Mingfei Han, Haoyu He, Xiaojun Chang, and Bohan Zhuang. 2024. Longvlm: Efficient long video understanding via large language models. In *European Conference on Computer Vision*. Springer, 453–470.

[64] Chu-Pan Wong, Yingfei Xiong, Hongyu Zhang, Dan Hao, Lu Zhang, and Hong Mei. 2014. Boosting bug-report-oriented fault localization with segmentation and stack-trace analysis. In *2014 IEEE international conference on software maintenance and evolution*. IEEE, 181–190.

[65] Mulong Xie, Sidong Feng, Zhenchang Xing, Jieshan Chen, and Chunyang Chen. 2020. UIED: a hybrid tool for GUI element detection. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1655–1659.

[66] Mulong Xie, Zhenchang Xing, Sidong Feng, Xiwei Xu, Liming Zhu, and Chunyang Chen. 2022. Psychologically-inspired, unsupervised inference of perceptual groups of GUI widgets from GUI images. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 332–343.

[67] Yanfu Yan, Nathan Cooper, Oscar Chaparro, Kevin Moran, and Denys Poshyvanyk. 2024. Semantic gui scene learning and video alignment for detecting duplicate video-based bug reports. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13.

[68] Jianwei Yang, Hao Zhang, Feng Li, Xueyan Zou, Chunyuan Li, and Jianfeng Gao. 2023. Set-of-mark prompting unleashes extraordinary visual grounding in gpt-4v. *arXiv preprint arXiv:2310.11441* (2023).

[69] Mingyue Yuan, Jieshan Chen, Zhenchang Xing, Aaron Quigley, Yuyu Luo, Tianqi Luo, Gelareh Mohammadi, Qinghua Lu, and Liming Zhu. 2024. DesignRepair: Dual-Stream Design Guideline-Aware Frontend Repair with Large Language Models. *arXiv preprint arXiv:2411.01606* (2024).

[70] Yue Zhao, Ishan Misra, Philipp Krähenbühl, and Rohit Girdhar. 2023. Learning video representations from large language models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 6586–6597.

[71] Yu Zhao, Tingting Yu, Ting Su, Yang Liu, Wei Zheng, Jingzhi Zhang, and William GJ Halfond. 2019. Recdroid: automatically reproducing android application crashes from bug reports. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 128–139.

[72] Ting Zhou, Yanjie Zhao, Xinyi Hou, Xiaoyu Sun, Kai Chen, and Haoyu Wang. 2025. DeclarUI: Bridging Design and Development with Automated Declarative UI Code Generation. *Proceedings of the ACM on Software Engineering* 2,

FSE (2025), 219–241.